Part A2: Language Comparison Report

End of Module Assignment – Launch into Computing (January 2026 B)

Introduction

JavaScript and Python are two of the most widely used programming languages in modern software development. Despite performing the same task, the provided scripts differ substantially in design philosophy, syntax, and the contexts for which each is best suited. This report examines those differences across four dimensions:

- programming model,
- readability and developer accessibility,
- error handling and debugging,
- and suitability for contemporary computing tasks.

Programming Model

Both Python and JavaScript are multi-paradigm languages that support procedural, object-oriented, and functional programming styles. However, their default orientations reflect their origins and intended use cases.

Python was designed with clarity and simplicity as primary goals. Its object model is consistent and uniform: everything in Python is an object, and the language encourages a straightforward procedural or object-oriented approach depending on the task (Downey, 2024). Python functions can be passed as values and used flexibly throughout a program, and the language provides a compact one-line syntax for transforming lists that keeps data processing code concise and readable. The

provided script demonstrates this directly: normalisation, deduplication, sorting, and formatting were each expressed in a single readable line.

JavaScript was originally designed as a scripting language for web browsers, and its programming model reflects that heritage. It supports prototypal inheritance rather than classical class-based inheritance, a distinction that Haverbeke (2024) describes as one of the most commonly misunderstood aspects of the language for developers arriving from other backgrounds. The introduction of the `class` keyword in ECMAScript 2015 provided more familiar syntax, but the underlying prototype chain remains. JavaScript is also inherently event-driven, built around asynchronous callbacks and, in modern usage, promises and async/await patterns — a model well suited to web interactions but which adds conceptual complexity absent from Python's predominantly synchronous execution model (Haverbeke, 2024).

Greenaway (2023) notes that the choice of programming model shapes how developers reason about problems. Python's consistency makes it easier to build a reliable mental model of how code executes, whereas JavaScript's flexibility — supporting multiple paradigms without enforcing any — can produce codebases that are structurally inconsistent across teams or projects.

Readability and Developer Accessibility

Python's readability advantage is well established. Its syntax enforces indentation as a structural element, eliminating the need for curly braces to denote code blocks. Keywords read closely to natural English, and the language's design philosophy (summarised in its internal document PEP 20 as "readability counts") prioritises explicitness over shortness(Havill, 2021). These properties make Python a

consistently recommended first language for developers entering the field (Havill, 2021; Downey, 2024).

JavaScript's syntax follows the C family of languages, using braces, semicolons, and a more verbose structure. Experienced developers often find this familiar, but the language contains several behaviours that can reduce readability for newcomers. For example, the `typeof` operator returns `"object"` for both arrays and `null`, which is counterintuitive and a common source of confusion. The `==` operator performs implicit type coercion, meaning it silently converts values before comparing them rather than checking type and value directly. Variable declarations using `var` are scoped to the function rather than the block, unlike the behaviour most developers expect. Haverbeke (2024) acknowledges these as historical artefacts. Later standards have partially addressed them, but they remain present in legacy codebases.These characteristics were directly encountered during the modification process: replacing `typeof names !== 'object'` with `!Array.isArray(names)` was necessary precisely because the simpler check produces misleading results.

In a professional engineering context, readability has a direct cost implication. Code that requires additional cognitive effort to interpret increases onboarding time, review overhead, and the likelihood of introducing defects during maintenance. Stephens (2022) identifies code readability as a foundational quality attribute in software engineering, noting that maintainability and readability are inseparable concerns in long-lived systems.

Error Handling and Debugging

Python's error handling is explicit and consistent. The `try/except` block, combined with typed exceptions such as `TypeError` and `ValueError`, allows errors to be caught, classified, and reported with precision. Python's interpreter produces descriptive stack traces that identify the exact line and nature of a failure, making debugging relatively straightforward (Downey, 2024).

JavaScript's error handling follows a similar `try/catch/finally` structure. However, the language's tolerance for implicit type coercion means that many operations that would raise an immediate exception in Python instead produce unexpected but valid-looking results. Adding a number to a string in JavaScript returns a string; in Python, it raises a `TypeError`. This permissiveness can allow defects to propagate silently through a system before eventually surfacing as incorrect output — precisely the behaviour observed in the original scripts, where the deduplication failure produced no error.

Asynchronous error handling in JavaScript adds further complexity. Errors thrown inside a callback or unhandled promise rejection do not surface in the same way as synchronous exceptions, requiring additional patterns such as `.catch()` chaining or `try/catch` within `async` functions (Haverbeke, 2024). For the simple synchronous task in the provided scripts, this distinction was not relevant, but it becomes significant in production applications.

Suitability for Modern Computing Tasks

The two languages occupy distinct but occasionally overlapping niches in modern software development.

Python has become the dominant language for data science, machine learning, and artificial intelligence. Its ecosystem (NumPy, Pandas, Scikit-learn, TensorFlow, and PyTorch, among others) provides tooling with no equivalent in JavaScript. Chang (2022) demonstrates this dominance employing Python-based machine learning and deep learning techniques across healthcare analytics tasks, including diagnosis, outcome prediction, and epidemiological modelling, providing evidence of how thoroughly Python underpins real-world AI implementation. Kalita et al. (2023) further establish that Python's combination of readable syntax and powerful numerical libraries makes it the practical default for data-intensive work. In professional engineering environments, Python is also widely used for test automation, infrastructure scripting, and CI/CD pipeline tooling — domains where its conciseness and library range are directly valuable.

JavaScript's primary domain remains the web. It is the only language that runs natively in the browser, making it indispensable for front-end development. Node.js has expanded its applicability to server-side development, and its event-driven model makes it well-suited to real-time applications such as chat systems, live dashboards, and high-volume APIs handling concurrent connections (Haverbeke, 2024). However, JavaScript's suitability for data science or systems programming remains limited compared to Python's ecosystem.

Conclusion

JavaScript and Python are capable, mature languages, but serve different purposes and embody different design priorities. Python rewards clarity, consistency, and short forms, making it the stronger choice for data science, automation, and tasks where maintainability is paramount. JavaScript's event-driven model and native browser

support make it essential for web development, though its implicit behaviours require greater developer discipline to manage safely. For the name-processing task examined in this report, Python produced a more readable and concise implementation; the observed structural differences reflect the broader characteristics of each language at scale.

References

Chang, V. (2022) *Novel AI and Data Science Advancements for Sustainability in the Era of COVID-19*. London: Academic Press. Available at: https://learning.oreilly.com/library/view/novel-ai-and/9780323903783/ (Accessed: 8 April 2026).

Downey, A.B. (2024) *Think Python*. 3rd edn. Sebastopol, CA: O'Reilly Media. Available at: https://learning.oreilly.com/library/view/think-python-3rd/9781098155421/ (Accessed: 3 April 2026).

Greenaway, J. (2023) *Fundamentals for Self-Taught Programmers: Embark on Your Software Engineering Journey without Exhaustive Courses and Bulky Tutorials* Birmingham: Packt Publishing Ltd. Available at: https://learning.oreilly.com/library/view/fundamentals-for-self-taught/9781801812115/ (Accessed: 3 April 2026).

Haverbeke, M. (2024) *Eloquent JavaScript*. 4th edn. San Francisco: No Starch Press. Available at: https://learning.oreilly.com/library/view/eloquent-javascript-4th/9781098182502/ (Accessed: 3 April 2026).

Havill, J. (2021) *Discovering Computer Science: Interdisciplinary Problems, Principles, and Python Programming*. 2nd edn. Abingdon: CRC Press. Available at: https://doi.org/10.1201/9781003037149

Kalita, J.K., Bhattacharyya, D.K. and Roy, S. (2023) *Fundamentals of Data Science: Theory and Practice*. Chantilly: Elsevier Science & Technology. Available at: https://doi.org/10.1016/C2021-0-01428-0

Stephens, R. (2022) *Beginning Software Engineering*. 2nd edn. Newark: John Wiley & Sons. Available at: https://learning.oreilly.com/library/view/beginning-software-engineering/9781119901709/ (Accessed: 8 April 2026).